Numerical methods in Fortran

Devendra Singh Bhakuni

dbhakuni@ictp.it

/afs/ictp/public/d/dbhakuni/NM_course/NM_lec1.pdf

November 14, 2025

Topics to be covered

- **Lec. 1**– Random number generators: middle square method, linear congruent method, built-in FORTRAN random number generators.

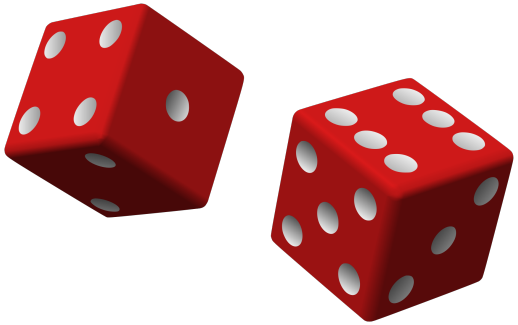
- **Lec. 2**– Non-uniform random numbers: transformation method, exponential and Gaussian random number generator

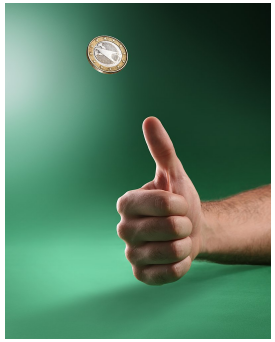
- **Lec. 3**– Rejection method and Monte-Carlo integration.

- **Lec. 4**– Random walk problem.

Random number generators and their uses

- Generates a sequence of numbers that can not be predicted.

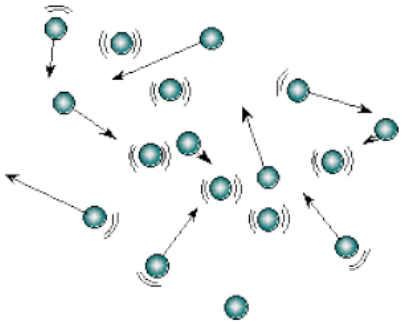


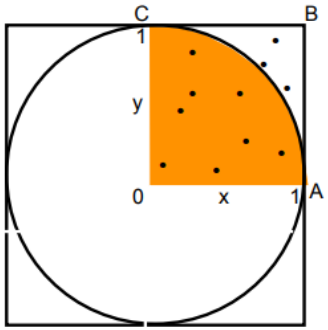


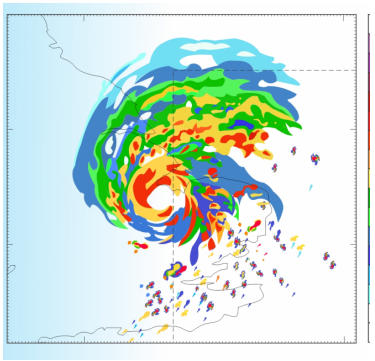
◇ Physical process that contains randomness: radioactive decay, thermal motion.

◇ Monte Carlo integration: multi-dimensional integrals.

◇ Simulation in Classical Statistical and Quantum Mechanics.







True random number generators

◇ Computer must use some external physical variable that is unpredictable, such as radioactive decay of isotopes and atmospheric noise.

◇ Truly random numbers are difficult to generate because they are not cost-efficient.

◇ Slow to generate, no reproducibility.

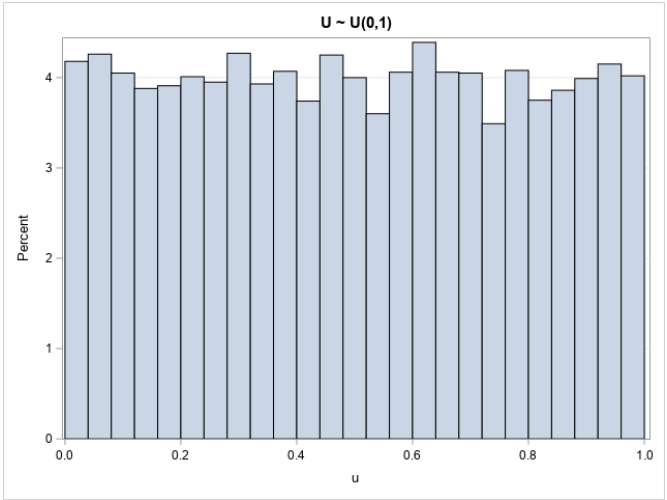
◇ Required for cryptography.

Pseudo random number generators

◇ Long sequences of numbers $(x_1, x_2, x_3, ...)$ generated by an algorithm on a computer. Look "nearly random" but completely deterministic!

◇ They have statistical properties similar to true random numbers and usually distributed uniformly in range $[0, 1]$: uniform distribution:

$$P(x) = \begin{cases} 1 & \text{if } 0 \leq x \leq 1, \\ 0 & \text{otherwise.} \end{cases} \qquad P(x) = \begin{cases} \frac{1}{(b-a)} & \text{if } a \leq x \leq b, \\ 0 & \text{otherwise.} \end{cases}$$



◇ The distribution is "normalized:"

$$\int_{-\infty}^{\infty} P(x)dx = 1$$

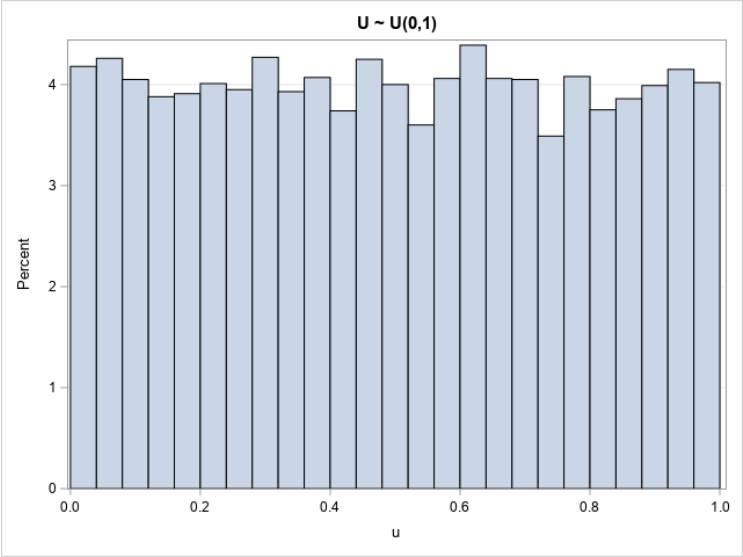
◇ To be precise, the algorithm generates integers I_n between 0 and M, and returns a real value. The real value is : $x_n = \frac{\text{float}(I_n)}{M}$.

Pseudo random numbers generation - Algorithms

Two among the simplest (and oldest) algorithms:

- ◇ The von Neumann method (Middle square algorithm)
- ◇ The Linear Congruential Method (LCM)

We will eventually use the in-built FORTRAN function to generate random numbers!

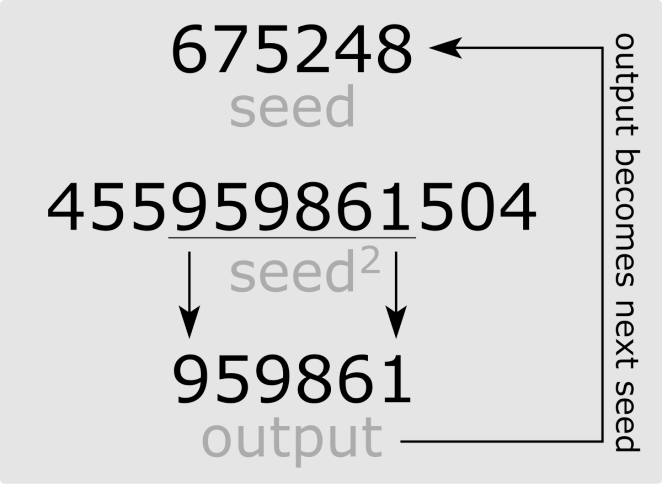


von Neumann (Middle square) algorithm

To generate a d-digit integer sequence:
- take any d digit number
- square it
- take the middle d digits of the result

loop it.

Limitation:
depending on the initial choice, you can be trapped into short loops.



Algorithm

Data: Seed X_0
Result: Random numbers
while *true* **do**
  $X_0 \leftarrow X_0^2$;
  Extract the middle digits as the random number;
  Output the random number;
end

Hint: To get the middle d digits: $M_1 = \frac{M}{10^{d/2}}$ and then $\mathrm{mod}(M_1, 10^d)$.

Linear Congruential Method (LCM)

The Linear Congruential Method is defined by the recurrence relation:

$$X_{n+1} = (a \cdot X_n + c) \mod m$$

Where:

X_n is the current pseudorandom number. X_0 - initial value (seed)

a is the multiplier. c is the increment.

m is the modulus.

$m - 1$ is the largest number.

Algorithm

Data: Seed X_0, Multiplier a, Increment c, Modulus m

Result: Random numbers

while *true* **do**

  $X_{n+1} = (a \cdot X_n + c) \mod m$;

  Output X_{n+1} as the random number;

end

Limitation of LCG

• A poor choice for the constants can lead to very poor sequences. During a finite period, the sequence repeats itself.

Example:

$$a = c = I_0 = 7 \text{ and } m = 10, \qquad \text{Sequence} : 7, 6, 9, 0, 7, 6, 9, 0, ...$$

• Overflow issue - multiplication of large numbers will be out of bounds.

• m should be as large as possible since the period can never be longer than m

• Typically, one usually chooses m to be near the largest integer that can be represented. On a 32 bit machine, that is $2^{31} \approx 2 \times 10^9$

• Good choice (a and m are coprime): $c = 0, m = 2^{31} - 1, a = 16807$

Using in-built FORTRAN functions

random_seed: Initializes or resets the pseudo-random number generator.

`call random_seed()` — uses a system-dependent default seed.

`call random_seed(put = seed_array)` — sets a specific seed for reproducibility. The `seed_array` must have exactly the length returned by `call random_seed(size = seed_size)`.

`call random_seed(get = seed_array)` — retrieves the current seed value into `seed_array`.

random_number: Generates random numbers uniformly distributed in the interval [0,1).

`call random_number(x)` — fills variable or array `x` with random values. Works for both scalar and array arguments.

• Other uniform distributions, uniform x in $[a, b]$: $x = a + (b - a)u$, where u is a random number in $[0, 1]$

Random Number Example in Fortran

```fortran
program testrand
  implicit none
  integer :: i, N, seed_size
  real(8), allocatable :: array_1d(:), array_2d(:,:)
  integer, allocatable :: seed(:)
  call random_seed(size = seed_size)
  allocate(seed(seed_size))
  seed = 12345 + (/ (i, i = 1, seed_size) /)
  call random_seed(put = seed)
  N = 100
  allocate(array_1d(N))
  call random_number(array_1d)
  allocate(array_2d(N, N))
  call random_number(array_2d)
  deallocate(array_1d)
  deallocate(array_2d)
  deallocate(seed)
end program testrand
```

Assignment-13

Consider the linear system

$$\dot{\mathbf{x}} = A\mathbf{x},$$

where

$$\mathbf{x} = \begin{pmatrix} x \\ y \end{pmatrix}, \quad A = \begin{pmatrix} a & b \\ c & d \end{pmatrix},$$

and where the numbers a, b, c, d are drawn independently and from a uniform distribution in the interval $[-1, 1]$. Generate 10^6 such random matrices, and print the fraction of these matrices that are saddle points, unstable nodes, stable nodes, stable spirals, unstable spirals.

Optional: Write subroutines for the middle square and the linear congruent method and generate random numbers between $[0, 10]$

Submit your code as ***surname_assignment_13.f90*** to ***dbhakuni@ictp.it*** before the next lesson.

Hints: Classification Criteria

Let $\tau = \text{trace}(A) = a + d$ and $\Delta = \det(A) = ad - bc$.
The eigenvalues $\lambda_{1,2}$ are the roots of $\lambda^2 - \tau\lambda + \Delta = 0$.

Type	**Conditions**	**Stability**
Stable Node	$\tau^2 - 4\Delta > 0$, $\tau < 0$, $\Delta > 0$	Stable
Unstable Node	$\tau^2 - 4\Delta > 0$, $\tau > 0$, $\Delta > 0$	Unstable
Saddle Point	$\Delta < 0$	Unstable
Stable Spiral	$\tau^2 - 4\Delta < 0$, $\tau < 0$	Stable
Unstable Spiral	$\tau^2 - 4\Delta < 0$, $\tau > 0$	Unstable